KFS2 — GDT & Stack

Guide super simple pour débutant · partie obligatoire uniquement

Projet Kernel From Scratch (42) · sujet version 2
Basé sur le code de ton KFS1 (boot.asm + kernel.c + linker.ld + Makefile)
Octobre 2026

Sommaire

1. Ce que le sujet demande (en une page)
2. Comprendre la mémoire et les segments
3. La GDT, c'est quoi exactement ?
4. Anatomie d'une entrée de GDT (8 octets)
5. L'octet d'accès et les flags, bit par bit
6. Les sélecteurs : comment le CPU utilise la GDT
7. Charger la GDT : lgdt et le « BIOS »
8. La pile (stack) du kernel
9. printk : un mini printf pour le kernel
10. Le code complet, fichier par fichier
11. Compiler, lancer, vérifier
12. Questions probables en soutenance
13. Checklist finale

1. Ce que le sujet demande (en une page)

KFS2 reprend ton KFS1 (le kernel qui affiche « 42 ») et lui ajoute deux choses :

Exigence du sujet	Ce que ça veut dire concrètement
Créer une **Global Descriptor Table**	Remplir un tableau de 7 cases de 8 octets qui décrit les zones mémoire (les « segments »).
Elle contient : Kernel Code, Kernel Data, Kernel Stack, User Code, User Data, User Stack	6 entrées + 1 entrée vide obligatoire au début (la « null entry ») = 7 entrées.
La GDT est à l'adresse `0x00000800`	Tu n'utilises pas une variable globale : tu écris directement tes 56 octets à l'adresse physique 0x800.
« Déclarer la GDT au BIOS »	En vrai on la déclare au **CPU** avec l'instruction assembleur `lgdt`, puis on recharge les registres de segment.
Un outil qui affiche la pile du kernel « de façon humaine »	Une fonction `print_kernel_stack()` qui fait un dump hexa + ASCII de la pile, depuis `esp` jusqu'au haut de la pile.
Conseil : faire un `printk`	Un mini `printf` (%d %x %s %p...) qui écrit dans le buffer VGA. Indispensable pour afficher des adresses.
Le travail ne doit pas dépasser 10 Mo	Ton `kfs.iso` doit rester petit (le tien fait quelques Mo, c'est bon).
Makefile, flags, i386, linker.ld perso	Identique à KFS1. On garde tout.

Le plan en 4 étapes

1. Ajouter `gdt.c` : construit les 7 entrées à 0x800 et appelle `gdt_flush`.
2. Ajouter `gdt_flush` dans `boot.asm` : `lgdt` + rechargement de CS, DS, ES, FS, GS, SS.
3. Ajouter `printk.c` : mini printf.
4. Ajouter `stack.c` : `print_kernel_stack()`.

2. Comprendre la mémoire et les segments

2.1 La RAM, c'est une grande liste d'octets

La mémoire est une suite d'octets numérotés de 0 jusqu'à (taille de la RAM - 1). Le numéro d'un octet s'appelle son **adresse**. En 32 bits, une adresse tient sur 32 bits, donc le CPU peut adresser 2^{32} octets = 4 Go maximum.

2.2 Pourquoi des « segments » ?

Dans les années 80, le processeur Intel 8086 avait des registres de 16 bits mais voulait adresser 1 Mo de mémoire. Intel a inventé la **segmentation** : une adresse est composée de deux morceaux, un **segment** (la zone) et un **offset** (le décalage dans la zone).

Avec le 80386 (notre i386, mode protégé 32 bits), les segments ont été améliorés : chaque segment a maintenant une **base** (où il commence), une **limite** (sa taille) et des **droits** (code exécutable ? données écrivables ? réservé au kernel ?).

> **Analogie**
> Imagine un immeuble (la RAM). Un segment, c'est un badge d'accès qui dit : « ce badge ouvre les étages 0 à 10, en lecture seule, réservé aux admins ». La GDT, c'est le tableau au rez-de-chaussée qui liste tous les badges existants.

2.3 Comment le CPU calcule une adresse

Quand ton code fait `mov eax, [0x1000]`, le CPU ne va pas directement à 0x1000. Il fait :

```
adresse réelle (linéaire) = base du segment (DS) + 0x1000
```

et il vérifie que 0x1000 est inférieur à la limite du segment, et que tu as le droit de lire. Sinon : faute de protection (#GP), le CPU plante.

Le CPU choisit le segment selon ce que tu fais :

Registre	Utilisé pour
`CS`	Code Segment : d'où viennent les instructions exécutées (eip)
`DS`	Data Segment : lecture/écriture de données par défaut
`SS`	Stack Segment : la pile (push, pop, call, ret, esp, ebp)
`ES, FS, GS`	Segments de données supplémentaires (instructions de copie de chaînes, etc.)

2.4 Le modèle « flat » (ce qu'on va faire)

Aujourd'hui plus personne ne découpe vraiment la mémoire en segments : la protection est faite par la **pagination** (KFS3). Mais le CPU **oblige** à avoir une GDT en mode protégé. La solution universelle (Linux fait pareil) : le modèle **flat**.

- Tous les segments ont **base = 0** et **limite = 4 Go** (toute la mémoire).
- Donc adresse linéaire = 0 + offset = offset. La segmentation devient « transparente ».
- Seuls les **droits** changent d'une entrée à l'autre : code ou data, ring 0 (kernel) ou ring 3 (user).

C'est pour ça que tes entrées « Kernel Data » et « Kernel Stack » auront exactement les mêmes valeurs : la différence est logique (on sait que SS pointe sur l'entrée « stack »), pas physique.

3. La GDT, c'est quoi exactement ?

La **GDT** (Global Descriptor Table) est simplement un **tableau en mémoire**. Chaque case fait **8 octets** et s'appelle un **descripteur** (descriptor). Un descripteur décrit un segment : base, limite, droits.

Notre GDT aura 7 cases, soit 7 × 8 = **56 octets**, placées à l'adresse **0x800** :

Index	Adresse	Sélecteur	Contenu	Droits
0	`0x800`	`0x00`	**Null** (obligatoire, tout à zéro)	aucun
1	`0x808`	`0x08`	**Kernel Code**	ring 0, exécutable + lisible
2	`0x810`	`0x10`	**Kernel Data**	ring 0, lisible + écrivable
3	`0x818`	`0x18`	**Kernel Stack**	ring 0, lisible + écrivable
4	`0x820`	`0x20`	**User Code**	ring 3, exécutable + lisible
5	`0x828`	`0x28`	**User Data**	ring 3, lisible + écrivable
6	`0x830`	`0x30`	**User Stack**	ring 3, lisible + écrivable

> **Pourquoi une entrée « null » ?**
> Intel impose que l'entrée 0 ne soit jamais utilisée. Si un registre de segment contient 0, le CPU lance une faute dès qu'on s'en sert. C'est une sécurité : un registre « non initialisé » (valeur 0) plante proprement au lieu d'accéder n'importe où.

3.1 Les rings (niveaux de privilège)

Le CPU x86 a 4 niveaux de privilège, de 0 (tout permis) à 3 (restreint). En pratique on n'en utilise que deux :

- **Ring 0** = le kernel. Peut tout faire : instructions privilégiées, accès au matériel.
- **Ring 3** = les programmes utilisateur. Ne peuvent pas toucher au matériel ni exécuter certaines instructions.

Le ring est stocké dans chaque descripteur (champ **DPL**). Pour l'instant ton kernel tourne entièrement en ring 0 ; les entrées « user » existent pour plus tard (quand tu lanceras des programmes), le sujet demande juste qu'elles soient présentes.

3.2 Pourquoi l'adresse 0x800 ?

La mémoire basse (le premier Mo) a une organisation historique :

Adresses	Contenu
`0x0000 – 0x03FF`	Table des interruptions du mode réel (IVT), inutile en mode protégé
`0x0400 – 0x04FF`	Zone de données du BIOS (BDA)
`0x0500 – 0x7BFF`	**Libre** : c'est là que le sujet nous fait mettre la GDT (0x800)
`0x7C00 – 0x7DFF`	Secteur de boot (bootloader)
`0x100000 (1 Mo)`	Ton kernel (c'est le `. = 1M` de ton linker.ld)

0x800 est donc une zone libre, basse, toujours présente, que GRUB n'utilise pas. Le sujet impose cette adresse pour que tu apprennes à écrire à une adresse physique précise au lieu de laisser le compilateur choisir.

4. Anatomie d'une entrée de GDT (8 octets)

C'est la partie la plus pénible : Intel a découpé la base et la limite en morceaux dispersés (pour rester compatible avec le 80286). Voici les 8 octets dans l'ordre où ils sont en mémoire :

Octet	0	1	2	3	4	5	6	7
Contenu	limite bits 0–15		base bits 0–15		base bits 16–23	accès	flags (4 bits) + limite bits 16–19	base bits 24–31
Champ C	limit_low		base_low		base_middle	access	granularity	base_high

Ce qui donne cette structure en C. `__attribute__((packed))` est **obligatoire** : sans lui gcc pourrait ajouter des octets de remplissage entre les champs, et le CPU lirait n'importe quoi.

```
struct gdt_entry {
    uint16_t limit_low;     /* limite, bits 0–15  */
    uint16_t base_low;      /* base,   bits 0–15  */
    uint8_t  base_middle;   /* base,   bits 16–23 */
    uint8_t  access;        /* droits */
    uint8_t  granularity;   /* flags (4 bits hauts) + limite bits 16–19 (4 bits bas) */
    uint8_t  base_high;     /* base,   bits 24–31 */
} __attribute__((packed));   /* taille garantie : 8 octets */
```

4.1 Base et limite dans le modèle flat

- **Base = 0** : tous les morceaux de base valent 0.
- **Limite = 0xFFFFF** (20 bits, tous à 1) avec le flag **G = 1** (granularité en pages de 4 Ko). Le CPU calcule : (0xFFFFF + 1) × 4096 = 4 Go. La limite est sur 20 bits seulement, c'est pour ça qu'on a besoin du flag G pour couvrir 4 Go.

4.2 La fonction qui remplit une entrée

On écrit une petite fonction qui découpe `base` et `limit` en morceaux avec des décalages (`>>`) et des masques (`&`) :

```
static void gdt_set_entry(int i, uint32_t base, uint32_t limit,
                          uint8_t access, uint8_t flags)
{
    gdt[i].limit_low   = limit & 0xFFFF;          /* 16 bits bas de la limite  */
    gdt[i].base_low    = base & 0xFFFF;           /* 16 bits bas de la base    */
    gdt[i].base_middle = (base >> 16) & 0xFF;     /* bits 16–23 de la base     */
    gdt[i].access      = access;
    gdt[i].granularity = ((limit >> 16) & 0x0F)   /* bits 16–19 de la limite   */
                       | ((flags & 0x0F) << 4);   /* 4 flags dans les bits hauts */
    gdt[i].base_high   = (base >> 24) & 0xFF;     /* bits 24–31 de la base     */
}
```

> **Rappel masques et décalages**
> `x & 0xFFFF` garde les 16 bits de droite. `x >> 16` jette les 16 bits de droite (on obtient les bits de gauche). Donc `(x >> 16) & 0xFF` = les bits 16 à 23 de x.

5. L'octet d'accès et les flags, bit par bit

5.1 L'octet d'accès (access)

Bit	7	6	5	4	3	2	1	0
Nom	P	DPL		S	E	DC	RW	A

Bit	Signification	Chez nous
P (Present)	1 = l'entrée est valide	toujours 1
DPL (2 bits)	niveau de privilège : 00 = ring 0, 11 = ring 3	00 pour kernel, 11 pour user
S	1 = segment code ou data (0 = segment système, TSS...)	toujours 1
E (Executable)	1 = segment de code, 0 = segment de données	1 pour code, 0 pour data/stack
DC	code : conforming ; data : direction (1 = grandit vers le bas)	toujours 0 (on reste simple)
RW	code : lisible ? data : écrivable ?	toujours 1
A (Accessed)	le CPU le met à 1 quand il utilise le segment	0 au départ

Les 4 valeurs à connaître :

Entrée	P	DPL	S	E	DC	RW	A	Binaire	Hexa
Kernel Code	1	00	1	1	0	1	0	`1001 1010`	**`0x9A`**
Kernel Data / Stack	1	00	1	0	0	1	0	`1001 0010`	**`0x92`**
User Code	1	11	1	1	0	1	0	`1111 1010`	**`0xFA`**
User Data / Stack	1	11	1	0	0	1	0	`1111 0010`	**`0xF2`**

> **Piège classique : le bit DC sur la pile**
> Certains tutos mettent DC = 1 (« expand-down ») sur le segment de pile parce que « la pile grandit vers le bas ». Avec une limite de 4 Go ça inverse la zone valide et tout plante. On laisse DC = 0 : la pile est un segment de données normal, et c'est `esp` qui descend, pas le segment.

5.2 Les 4 flags (moitié haute de l'octet granularity)

Bit	7	6	5	4
Nom	G	D/B	L	AVL
Chez nous	1	1	0	0

- **G** (Granularity) : 1 = la limite se compte en pages de 4 Ko (nécessaire pour atteindre 4 Go).
- **D/B** : 1 = segment 32 bits (0 = 16 bits).
- **L** : 1 = code 64 bits. On est en 32 bits, donc 0.
- **AVL** : libre pour l'OS, 0.

Flags = `1100` = **0xC**. Combiné aux 4 bits hauts de la limite (0xF), l'octet granularity vaut `0xCF` pour toutes les entrées sauf la null.

Au final, en mémoire à 0x808 (Kernel Code), on trouve exactement ces 8 octets : `FF FF 00 00 00 9A CF 00`. Tu pourras le vérifier dans QEMU (section 11).

6. Les sélecteurs : comment le CPU utilise la GDT

Un registre de segment (CS, DS, SS...) ne contient pas une adresse mais un **sélecteur** de 16 bits qui dit « prends l'entrée n° X de la GDT ».

Bits	15 ... 3	2	1-0
Nom	Index (numéro de l'entrée)	TI	RPL
Sens	index de la GDT	0 = GDT, 1 = LDT	ring demandé

Comme l'index commence au bit 3, un sélecteur vaut tout simplement **index × 8** (quand TI = 0 et RPL = 0) :

- Kernel Code = entrée 1 → sélecteur `0x08`
- Kernel Data = entrée 2 → sélecteur `0x10`
- Kernel Stack = entrée 3 → sélecteur `0x18`
- User Code = entrée 4 → `0x20`, User Data = 5 → `0x28`, User Stack = 6 → `0x30`

Le CPU fait : adresse de l'entrée = base de la GDT (0x800) + sélecteur. Par exemple `0x800 + 0x10 = 0x810` = Kernel Data. C'est littéralement un indice dans un tableau.

> **Ce qu'on mettra dans les registres**
> `CS = 0x08` (Kernel Code), `DS = ES = FS = GS = 0x10` (Kernel Data), `SS = 0x18` (Kernel Stack). Les sélecteurs user (0x20, 0x28, 0x30) ne sont pas utilisés pour l'instant.

7. Charger la GDT : lgdt et le « BIOS »

7.1 Le registre GDTR

Le CPU a un registre spécial, **GDTR**, qui mémorise où est la GDT. On le remplit avec l'instruction `lgdt` (Load GDT) à partir d'une petite structure de 6 octets :

```
struct gdt_ptr {
    uint16_t limit;  /* taille de la table EN OCTETS, MOINS 1 : 7*8 - 1 = 55 = 0x37 */
    uint32_t base;   /* adresse de la table : 0x800 */
} __attribute__((packed));
```

Le « moins 1 » est une convention Intel : la limite est l'offset du **dernier octet valide**.

7.2 « Déclarer au BIOS » ?

Le sujet dit « déclarer la GDT au BIOS ». C'est une formulation approximative : le BIOS ne sait rien de ta GDT. En réalité tu la déclares au **processeur** avec `lgdt`. GRUB avait installé sa propre GDT

temporaire pour te passer en mode protégé ; toi tu la remplaces par la tienne. Si on te pose la question en soutenance, réponds ça.

7.3 Pourquoi recharger les registres de segment ?

`lgdt` change seulement l'adresse de la table. Les registres CS, DS, SS... gardent leur ancienne valeur (celle de GRUB), et le CPU garde en cache les anciens descripteurs. Il faut donc **réécrire chaque registre** pour forcer le CPU à relire nos nouvelles entrées :

- DS, ES, FS, GS, SS : un simple `mov` suffit (`mov ax, 0x10 ; mov ds, ax`).
- **CS** : impossible avec `mov`. La seule façon est un **saut lointain** (far jump) : `jmp 0x08:suite`. Le CPU charge 0x08 dans CS et continue à l'étiquette `suite`.

Tout ça se fait en assembleur (dans `boot.asm`), car le C ne peut pas manipuler les registres de segment :

```
gdt_flush:
    mov eax, [esp + 4]   ; 1er argument : adresse de la struct gdt_ptr
    lgdt [eax]           ; dit au CPU : "ta GDT est là"

    mov ax, 0x10         ; sélecteur kernel data
    mov ds, ax
    mov es, ax
    mov fs, ax
    mov gs, ax

    mov ax, 0x18         ; sélecteur kernel stack
    mov ss, ax

    jmp 0x08:.flush      ; far jump : seule façon de recharger CS (kernel code)
.flush:
    ret
```

`[esp + 4]` : en C sur i386, les arguments d'une fonction sont poussés sur la pile. À l'entrée de la fonction, `[esp]` contient l'adresse de retour et `[esp + 4]` le premier argument.

8. La pile (stack) du kernel

8.1 C'est quoi ?

La pile est une zone mémoire où le CPU empile et dépile des valeurs de 4 octets (en 32 bits). Deux opérations : **push** (pose une valeur sur le dessus) et **pop** (retire la dernière posée). Dernière entrée, première sortie : **LIFO**.

Le registre **esp** (stack pointer) contient toujours l'adresse du dessus de la pile. Particularité sur x86 : **la pile grandit vers les adresses basses**. `push` fait `esp = esp - 4` puis écrit ; `pop` lit puis fait `esp = esp + 4`.

8.2 À quoi elle sert

- **Appels de fonction** : `call` pousse l'adresse de retour, `ret` la dépile et y saute.
- **Arguments** des fonctions (convention cdecl sur i386).
- **Variables locales** : `char msg[] = "Hello"` dans une fonction vit sur la pile.
- **Sauvegarde de registres**, notamment `ebp` (le « frame pointer »).

8.3 Notre pile : 16 Ko dans boot.asm

Tu l'avais déjà en KFS1. On ajoute juste `global` pour que le C puisse voir les deux bornes :

```
section .bss
align 16
global stack_bottom
global stack_top
stack_bottom:
    resb 16384        ; 16 Ko réservés (non initialisés)
stack_top:

_start:
    mov esp, stack_top   ; la pile démarre EN HAUT et descend
```

stack_top	adresse la plus haute. `esp` y démarre. Pile vide = esp == stack_top.
esp	descend à chaque push / call / variable locale.
stack_bottom	adresse la plus basse. Si esp passe en dessous : débordement de pile (stack overflow).

8.4 Ce qu'on voit dans un dump de pile

Quand `_start` fait `call kmain`, puis que `kmain` appelle `print_kernel_stack`, la pile ressemble à ça (adresses hautes en bas du dessin, comme en mémoire) :

Adresse	Contenu	Mis par
`esp (bas)`	variables locales de print_kernel_stack (esp, ebp, p, i)	print_kernel_stack
`...`	ancien ebp (celui de kmain)	prologue de print_kernel_stack
`...`	adresse de retour dans kmain	le `call print_kernel_stack`
`...`	variables locales de kmain : msg = "Hello KFS2", magic = 0xDEADBEEF	kmain
`...`	ancien ebp (zéro, GRUB)	prologue de kmain
`stack_top - 4`	adresse de retour dans _start (le `.hang`)	le `call kmain`
`stack_top`	(fin, pile vide)	

C'est exactement ce que `print_kernel_stack()` affiche : on lit la mémoire depuis `esp` jusqu'à `stack_top`, 16 octets par ligne, en hexa puis en ASCII. Le texte « Hello KFS2 » et les octets `ef be ad de` (0xDEADBEEF écrit en little-endian) sont visibles dans le dump : c'est une bonne démo pour la soutenance.

> **Little-endian**
> Sur x86, un entier de 4 octets est stocké octet de poids faible en premier. 0xDEADBEEF en mémoire = `EF BE AD DE`. C'est normal de le voir « à l'envers » dans le dump.

8.5 Lire esp et ebp depuis le C

Le C ne connaît pas les registres. On utilise l'assembleur inline de gcc :

```
uint32_t esp;
__asm__ volatile ("mov %%esp, %0" : "=r"(esp));   /* copie esp dans la variable C */
```

`"=r"(esp)` veut dire « mets le résultat dans un registre, puis dans ma variable esp ». `%%esp` est le vrai registre (double % car syntaxe AT&T de gcc).

9. printk : un mini printf pour le kernel

Tu n'as pas de libc, donc pas de `printf`. On en écrit un petit : `printk(const char *fmt, ...)`. Il parcourt la chaîne, affiche les caractères normaux avec ton `putchar` de KFS1, et quand il voit `%` il regarde la lettre suivante :

Format	Affiche	Exemple
`%d`	entier signé en décimal	`-42`
`%u`	entier non signé en décimal	`16384`
`%x`	hexadécimal	`deadbeef`
`%02x / %08x`	hexadécimal complété par des zéros à gauche	`0a / 00000800`
`%p`	pointeur : 0x + 8 chiffres hexa	`0x00000800`
`%s`	chaîne	`Hello`
`%c`	un caractère	`A`
`%%`	le caractère %	`%`

9.1 Les arguments variables : stdarg.h

Les `...` sont gérés par `va_list`, `va_start`, `va_arg`, `va_end`. Ils viennent de `<stdarg.h>`, qui est fourni par **gcc lui-même** (pas par la libc), donc autorisé même avec `-nostdlib`.

```
va_list args;
va_start(args, fmt);          /* démarre après le dernier argument nommé */
int n = va_arg(args, int);    /* prend l'argument suivant, en le lisant comme un int */
va_end(args);
```

9.2 Convertir un nombre en texte

Pour afficher 255 en hexa : on divise par 16 en boucle et on garde les restes. 255 / 16 = 15 reste 15 (f), 15 / 16 = 0 reste 15 (f). Les chiffres sortent à l'envers (« ff » ici, mais pour 256 on aurait 0,0,1), donc on les stocke dans un tampon puis on les affiche de la fin vers le début. C'est ce que fait `print_unsigned()`.

10. Le code complet, fichier par fichier

Arborescence finale (les fichiers en gras sont nouveaux ou modifiés) :

```
kfs2/
  Makefile            (modifié : 4 fichiers C au lieu d'un)
  linker.ld           (inchangé)
  Dockerfile, docker-compose.yml (inchangés)
  boot/boot.asm       (modifié : global stack_*, + gdt_flush)
  kernel/kernel.h     (nouveau : types + prototypes)
  kernel/kernel.c     (modifié : \n, scroll, kmain)
  kernel/gdt.c        (nouveau)
  kernel/printk.c     (nouveau)
  kernel/stack.c      (nouveau)
  iso/boot/grub/grub.cfg (menuentry renommée KFS_2)
```

10.1 `boot/boot.asm`

```
BITS 32

; ---------------------------------------------------------------
; Header Multiboot 1 (inchangé depuis KFS1)
; ---------------------------------------------------------------

section .multiboot
align 4

MB_MAGIC    equ 0x1BADB002   ; valeur que GRUB cherche pour reconnaître un kernel multiboot
MB_FLAGS    equ 0            ; aucune option demandée à GRUB
MB_CHECKSUM equ -(MB_MAGIC + MB_FLAGS) ; magic + flags + checksum doit faire 0

    dd MB_MAGIC
    dd MB_FLAGS
    dd MB_CHECKSUM


; ---------------------------------------------------------------
; Pile du kernel : 16 Ko réservés dans .bss
; stack_bottom / stack_top sont exportés (global) pour que le C
; puisse les lire et afficher la pile.
; ---------------------------------------------------------------

section .bss
align 16

global stack_bottom
global stack_top

stack_bottom:
    resb 16384
stack_top:


; ---------------------------------------------------------------
; Point d'entrée du kernel
; ---------------------------------------------------------------

section .text
global _start
extern kmain

_start:
```

```
    cli                 ; coupe les interruptions, rien n'est encore prêt pour les gérer
    mov esp, stack_top  ; la pile du host n'est pas garantie valide, on pose la nôtre
    call kmain          ; saute dans le code C

.hang:
    hlt                 ; si kmain revient, on arrête le CPU proprement
    jmp .hang


; ---------------------------------------------------------------
; gdt_flush(struct gdt_ptr *ptr)
; Charge notre GDT dans le CPU (lgdt) puis recharge tous les
; registres de segment pour qu'ils pointent sur nos nouvelles entrées.
; Sélecteurs = index * 8 :
;   0x08 = kernel code, 0x10 = kernel data, 0x18 = kernel stack
; ---------------------------------------------------------------

global gdt_flush

gdt_flush:
    mov eax, [esp + 4]  ; 1er argument : adresse de la struct gdt_ptr
    lgdt [eax]          ; dit au CPU : "ta GDT est là"

    mov ax, 0x10        ; sélecteur kernel data
    mov ds, ax
    mov es, ax
    mov fs, ax
    mov gs, ax

    mov ax, 0x18        ; sélecteur kernel stack
    mov ss, ax

    jmp 0x08:.flush     ; far jump : seule façon de recharger CS (kernel code)
.flush:
    ret

section .note.GNU-stack noalloc noexec nowrite progbits
```

10.2 kernel/kernel.h

```
#ifndef KERNEL_H
#define KERNEL_H

/* stdarg.h vient du compilateur (pas de la libc), donc autorisé en freestanding.
 * Il sert pour les fonctions à nombre d'arguments variable (printk). */
#include <stdarg.h>

/* Types kernel : pas de libc ici, donc on définit nous-mêmes les tailles fixes (i386) */
typedef unsigned char  uint8_t;
typedef unsigned short uint16_t;
typedef unsigned int   uint32_t;
typedef unsigned int   size_t;

/* kernel.c : écran VGA */
void clear_screen(void);
void putchar(char c);
void print(const char *s);

/* printk.c */
void printk(const char *fmt, ...);

/* gdt.c */
void gdt_init(void);

/* stack.c */
```

```
void print_kernel_stack(void);

#endif
```

10.3 kernel/gdt.c

```
#include "kernel.h"

/* Une entrée de GDT = 8 octets, dans l'ordre bizarre imposé par Intel.
 * __attribute__((packed)) interdit à gcc d'ajouter du padding entre les champs. */
struct gdt_entry {
    uint16_t limit_low;     /* limite, bits 0-15  */
    uint16_t base_low;      /* base,   bits 0-15  */
    uint8_t  base_middle;   /* base,   bits 16-23 */
    uint8_t  access;        /* droits : présent, ring, code/data, lecture/écriture */
    uint8_t  granularity;   /* 4 bits de flags (G, D/B) + limite bits 16-19 */
    uint8_t  base_high;     /* base,   bits 24-31 */
} __attribute__((packed));

/* Ce qu'on donne à l'instruction lgdt : la taille de la table - 1, et son adresse */
struct gdt_ptr {
    uint16_t limit;
    uint32_t base;
} __attribute__((packed));

#define GDT_ADDRESS 0x00000800  /* imposé par le sujet */
#define GDT_SIZE    7           /* null + 3 kernel + 3 user */

/* Octets d'accès (voir le PDF pour le détail bit par bit) */
#define ACCESS_KERNEL_CODE 0x9A /* 1001 1010 : présent, ring 0, code, lisible   */
#define ACCESS_KERNEL_DATA 0x92 /* 1001 0010 : présent, ring 0, data, écrivable */
#define ACCESS_USER_CODE   0xFA /* 1111 1010 : présent, ring 3, code, lisible   */
#define ACCESS_USER_DATA   0xF2 /* 1111 0010 : présent, ring 3, data, écrivable */

/* Flags : G=1 (limite en pages de 4 Ko), D/B=1 (segment 32 bits) */
#define FLAGS_32BIT_4K     0xC  /* 1100 */

/* Sélecteurs = index * 8 (utilisés dans boot.asm) :
 * 0x00 null  | 0x08 kcode | 0x10 kdata | 0x18 kstack
 * 0x20 ucode | 0x28 udata | 0x30 ustack */

static struct gdt_entry *gdt = (struct gdt_entry *)GDT_ADDRESS;
static struct gdt_ptr    gp;

extern void gdt_flush(struct gdt_ptr *ptr); /* boot.asm */

static void gdt_set_entry(int i, uint32_t base, uint32_t limit,
                          uint8_t access, uint8_t flags)
{
    gdt[i].limit_low   = limit & 0xFFFF;
    gdt[i].base_low    = base & 0xFFFF;
    gdt[i].base_middle = (base >> 16) & 0xFF;
    gdt[i].access      = access;
    gdt[i].granularity = ((limit >> 16) & 0x0F) | ((flags & 0x0F) << 4);
    gdt[i].base_high   = (base >> 24) & 0xFF;
}

void gdt_init(void)
{
    /* Modèle "flat" : chaque segment couvre toute la mémoire
     * (base 0, limite 4 Go). Seuls les droits changent d'une entrée à l'autre. */
    gdt_set_entry(0, 0, 0,       0,                   0);              /* null (obligatoire) */
    gdt_set_entry(1, 0, 0xFFFFF, ACCESS_KERNEL_CODE, FLAGS_32BIT_4K); /* kernel code  */
    gdt_set_entry(2, 0, 0xFFFFF, ACCESS_KERNEL_DATA, FLAGS_32BIT_4K); /* kernel data  */
    gdt_set_entry(3, 0, 0xFFFFF, ACCESS_KERNEL_DATA, FLAGS_32BIT_4K); /* kernel stack */
```

```
    gdt_set_entry(4, 0, 0xFFFFF, ACCESS_USER_CODE,   FLAGS_32BIT_4K); /* user code    */
    gdt_set_entry(5, 0, 0xFFFFF, ACCESS_USER_DATA,   FLAGS_32BIT_4K); /* user data    */
    gdt_set_entry(6, 0, 0xFFFFF, ACCESS_USER_DATA,   FLAGS_32BIT_4K); /* user stack   */

    gp.limit = sizeof(struct gdt_entry) * GDT_SIZE - 1; /* 7 * 8 - 1 = 55 = 0x37 */
    gp.base  = GDT_ADDRESS;

    gdt_flush(&gp); /* lgdt + rechargement des registres de segment */
}
```

10.4 kernel/printk.c

```
#include "kernel.h"

/* Affiche un nombre non signé dans une base donnée.
 * width > 0 : complète avec des zéros à gauche (ex: %02x, %08x) */
static void print_unsigned(uint32_t n, uint32_t base, int width)
{
    const char *digits = "0123456789abcdef";
    char        buf[32];
    int         i = 0;

    if (n == 0)
        buf[i++] = '0';
    while (n > 0) {
        buf[i++] = digits[n % base];
        n /= base;
    }
    while (i < width)
        buf[i++] = '0';
    while (i > 0)
        putchar(buf[--i]); /* les chiffres ont été calculés à l'envers */
}

static void print_signed(int n)
{
    if (n < 0) {
        putchar('-');
        print_unsigned((uint32_t)0 - (uint32_t)n, 10, 0);
    } else
        print_unsigned((uint32_t)n, 10, 0);
}

/* Mini printf du kernel. Formats : %d %u %x %p %c %s %%
 * On peut préciser une largeur avec des zéros : %02x, %08x */
void printk(const char *fmt, ...)
{
    va_list args;
    int     width;

    va_start(args, fmt);
    while (*fmt) {
        if (*fmt != '%') {
            putchar(*fmt++);
            continue;
        }
        fmt++;                          /* saute le '%' */
        width = 0;
        if (*fmt == '0') {              /* ex: "%08x" -> width = 8 */
            fmt++;
            while (*fmt >= '0' && *fmt <= '9')
                width = width * 10 + (*fmt++ - '0');
        }
        if (*fmt == '\0')
            break;
```

```
        switch (*fmt) {
            case 'd': print_signed(va_arg(args, int));                    break;
            case 'u': print_unsigned(va_arg(args, uint32_t), 10, width);  break;
            case 'x': print_unsigned(va_arg(args, uint32_t), 16, width);  break;
            case 'p': print("0x");
                      print_unsigned(va_arg(args, uint32_t), 16, 8);      break;
            case 'c': putchar((char)va_arg(args, int));                   break;
            case 's': print(va_arg(args, const char *));                  break;
            case '%': putchar('%');                                       break;
            default : putchar('%'); putchar(*fmt);                        break;
        }
        fmt++;
    }
    va_end(args);
}
```

10.5 kernel/stack.c

```
#include "kernel.h"

/* Symboles définis dans boot.asm : les deux bouts de la pile du kernel */
extern uint8_t stack_bottom[];
extern uint8_t stack_top[];

/* Affiche la pile du kernel de façon lisible :
 *   adresse : 16 octets en hexa | version ASCII |
 * On part de esp (le "haut" actuel de la pile) jusqu'à stack_top. */
void print_kernel_stack(void)
{
    uint32_t esp;
    uint32_t ebp;
    uint8_t *p;

    __asm__ volatile ("mov %%esp, %0" : "=r"(esp));
    __asm__ volatile ("mov %%ebp, %0" : "=r"(ebp));

    printk("=== Kernel stack ===\n");
    printk("stack_bottom = %p   stack_top = %p   taille  = %u octets\n",
           stack_bottom, stack_top, (uint32_t)(stack_top - stack_bottom));
    printk("esp          = %p   ebp       = %p   utilise = %u octets\n",
           esp, ebp, (uint32_t)stack_top - esp);

    p = (uint8_t *)esp;
    while (p < stack_top) {
        printk("%p: ", p);
        for (int i = 0; i < 16; i++) {
            if (p + i < stack_top)
                printk("%02x ", p[i]);
            else
                print("   ");
        }
        print(" |");
        for (int i = 0; i < 16 && p + i < stack_top; i++) {
            char c = (char)p[i];
            putchar((c >= 32 && c <= 126) ? c : '.');
        }
        print("|\n");
        p += 16;
    }
}
```

10.6 `kernel/kernel.c`

Par rapport à KFS1 : `putchar` gère `\n` et fait défiler l'écran quand il est plein, et `kmain` appelle `gdt_init()` puis `print_kernel_stack()`.

```
#include "kernel.h"

#define VGA_ADDRESS 0xB8000 /* adresse fixe du buffer texte VGA */
#define VGA_WIDTH   80
#define VGA_HEIGHT  25
#define VGA_COLOR   0x0F     /* blanc sur fond noir */

static uint16_t *const vga = (uint16_t *)VGA_ADDRESS;
static int cursor = 0; /* position du prochain caractère */

/* Lib de base : -nostdlib enlève la libc, et gcc peut appeler memset/memcpy tout seul */

size_t strlen(const char *s)
{
    size_t len = 0;

    while (s[len] != '\0')
        len++;
    return len;
}

void *memset(void *dst, int value, size_t n)
{
    uint8_t *bytes = (uint8_t *)dst;

    for (size_t i = 0; i < n; i++)
        bytes[i] = (uint8_t)value;
    return dst;
}

void *memcpy(void *dst, const void *src, size_t n)
{
    uint8_t *to = (uint8_t *)dst;
    const uint8_t *from = (const uint8_t *)src;

    for (size_t i = 0; i < n; i++)
        to[i] = from[i];
    return dst;
}

void clear_screen(void)
{
    for (int i = 0; i < VGA_WIDTH * VGA_HEIGHT; i++)
        vga[i] = (VGA_COLOR << 8) | ' '; /* case VGA = (couleur << 8) | caractère */
    cursor = 0;
}

/* Quand l'écran est plein : on remonte tout d'une ligne et on vide la dernière */
static void scroll(void)
{
    memcpy(vga, vga + VGA_WIDTH, (VGA_HEIGHT - 1) * VGA_WIDTH * 2);
    for (int i = 0; i < VGA_WIDTH; i++)
        vga[(VGA_HEIGHT - 1) * VGA_WIDTH + i] = (VGA_COLOR << 8) | ' ';
    cursor = (VGA_HEIGHT - 1) * VGA_WIDTH;
}

void putchar(char c)
{
    if (c == '\n')
        cursor = (cursor / VGA_WIDTH + 1) * VGA_WIDTH; /* début de la ligne suivante */
    else
```

```
        vga[cursor++] = (VGA_COLOR << 8) | (unsigned char)c;
    if (cursor >= VGA_WIDTH * VGA_HEIGHT)
        scroll();
}

void print(const char *s)
{
    while (*s)
        putchar(*s++);
}

void kmain(void)
{
    /* Variables locales : elles vivent sur la pile, on les retrouvera dans le dump */
    char     msg[]  = "Hello KFS2";
    uint32_t magic  = 0xDEADBEEF;

    clear_screen();
    gdt_init();                     /* 1) construit la GDT à 0x800, la charge dans le CPU */

    printk("42 - KFS2\n");
    printk("GDT chargee a l'adresse %p (%d entrees, %d octets)\n", 0x800, 7, 7 * 8);
    printk("msg = \"%s\"  magic = 0x%x\n\n", msg, magic);

    print_kernel_stack();           /* 2) affiche la pile du kernel */

    while (1)
        __asm__ volatile ("hlt"); /* le kernel n'a rien d'autre à faire, on boucle */
}
```

10.7 Makefile

Seul changement : la liste des sources C et une règle générique `kernel/%.o: kernel/%.c`.

```
NAME = kernel.elf
ISO = kfs.iso

ASM = nasm
CC = gcc
LD = ld

ASMFLAGS = -f elf32
# -ffreestanding -fno-builtin -nostdlib -nodefaultlibs : aucune dépendance à la libc de l'host
CFLAGS = -m32 -ffreestanding -fno-builtin -fno-stack-protector \
         -fno-pie -fno-pic -nostdlib -nodefaultlibs -Wall -Wextra
LDFLAGS = -m elf_i386 -T linker.ld -nostdlib

BOOT_SRC = boot/boot.asm
KERNEL_SRC = kernel/kernel.c kernel/gdt.c kernel/printk.c kernel/stack.c

BOOT_OBJ = boot/boot.o
KERNEL_OBJ = $(KERNEL_SRC:.c=.o)

ISO_DIR = iso
ISO_KERNEL = $(ISO_DIR)/boot/kernel.elf
GRUB_CFG = $(ISO_DIR)/boot/grub/grub.cfg

.PHONY: all iso run clean fclean re

all: $(NAME)

$(BOOT_OBJ): $(BOOT_SRC)
        $(ASM) $(ASMFLAGS) $< -o $@

kernel/%.o: kernel/%.c kernel/kernel.h
```

```
	$(CC) $(CFLAGS) -c $< -o $@

$(NAME): $(BOOT_OBJ) $(KERNEL_OBJ) linker.ld
	$(LD) $(LDFLAGS) -o $@ $(BOOT_OBJ) $(KERNEL_OBJ)

iso: $(ISO)

$(ISO): $(NAME) $(GRUB_CFG)
	mkdir -p $(ISO_DIR)/boot/grub
	cp $(NAME) $(ISO_KERNEL)
	grub-mkrescue -o $(ISO) $(ISO_DIR)

run: $(ISO)
	qemu-system-i386 -cdrom kfs.iso -m 32M -boot order=d,menu=off -nic none

clean:
	rm -f $(BOOT_OBJ) $(KERNEL_OBJ)

fclean: clean
	rm -f $(NAME) $(ISO) $(ISO_KERNEL)

re: fclean all
```

11. Compiler, lancer, vérifier

11.1 Compiler et créer l'ISO (dans Docker, comme en KFS1)

```
docker compose run --rm kfs bash -c "make re && make iso"
ls -la kfs.iso      # doit faire quelques Mo, bien moins que 10 Mo
```

11.2 Lancer dans QEMU et voir l'écran

```
docker compose run --rm kfs bash -c "qemu-system-i386 -cdrom kfs.iso -m 32M \
    -boot order=d,menu=off -nic none -display curses"
```

Tu dois voir à l'écran (capture réelle obtenue avec le code de ce guide ; les adresses exactes peuvent varier légèrement selon ta version de gcc) :

```
42 - KFS2
GDT chargee a l'adresse 0x00000800 (7 entrees, 56 octets)
msg = "Hello KFS2"  magic = 0xdeadbeef

=== Kernel stack ===
stack_bottom = 0x00104000   stack_top = 0x00108000   taille  = 16384 octets
esp          = 0x00107fb0   ebp       = 0x00107fd8   utilise = 80 octets
0x00107fb0: 00 00 01 00 d8 7f 10 00 dc 7f 10 00 d8 7f 10 00  |................|
0x00107fc0: b0 7f 10 00 10 00 00 00 08 00 00 00 c0 7f 10 00  |................|
0x00107fd0: 5e 20 10 00 e1 7f 10 00 f8 7f 10 00 76 12 10 00  |^ ..........v...|
0x00107fe0: 00 48 65 6c 6c 6f 20 4b 46 53 32 00 ef be ad de  |.Hello KFS2.....|
0x00107ff0: 00 00 00 00 00 00 00 00 00 00 00 00 0b 10 10 00  |................|
```

Lecture du dump : `ef be ad de` = 0xDEADBEEF en little-endian, `Hello KFS2` est visible dans la colonne ASCII, et les derniers 4 octets avant `stack_top` sont l'adresse de retour vers `_start` (dans la zone 0x0010xxxx = le code du kernel à 1 Mo).

11.3 Prouver que la GDT est bien à 0x800 (moniteur QEMU)

QEMU a un moniteur de debug. Avec `-monitor stdio` on tape des commandes dans le terminal :

```
docker compose run --rm kfs bash -c "qemu-system-i386 -cdrom kfs.iso -m 32M \
    -boot order=d -nic none -display none -monitor stdio"
(qemu) info registers
(qemu) xp /7xg 0x800
(qemu) quit
```

Résultat obtenu avec le code de ce guide :

```
(qemu) info registers
EAX=00000000 EBX=00010000 ECX=000003bf EDX=0000000c
ESI=00000000 EDI=00000000 EBP=00107ff8 ESP=00107fe0
EIP=00101277 EFL=00000046 [---Z-P-] CPL=0 II=0 A20=1 SMM=0 HLT=1
ES =0010 00000000 ffffffff 00cf9300 DPL=0 DS   [-WA]
CS =0008 00000000 ffffffff 00cf9a00 DPL=0 CS32 [-R-]
SS =0018 00000000 ffffffff 00cf9300 DPL=0 DS   [-WA]
DS =0010 00000000 ffffffff 00cf9300 DPL=0 DS   [-WA]
FS =0010 00000000 ffffffff 00cf9300 DPL=0 DS   [-WA]
GS =0010 00000000 ffffffff 00cf9300 DPL=0 DS   [-WA]
GDT=     00000800 00000037
```

```
(qemu) xp /7xg 0x800
0000000000000800: 0x0000000000000000 0x00cf9a000000ffff
0000000000000810: 0x00cf93000000ffff 0x00cf93000000ffff
0000000000000820: 0x00cffa000000ffff 0x00cff2000000ffff
0000000000000830: 0x00cff2000000ffff
```

- `GDT= 00000800 00000037` : base 0x800, limite 0x37 = 55 = 7 × 8 - 1. Exactement ce que demande le sujet.
- `CS =0008`, `DS =0010`, `SS =0018` : nos sélecteurs. QEMU affiche aussi pour chaque registre la base `00000000`, la limite `ffffffff` et les flags décodés (`DPL=0`, `CS32`, `DS` ...).
- Tu as mis `0x92` pour Kernel Data mais QEMU affiche `93` : c'est normal. Le CPU a mis le bit **A** (Accessed, bit 0) à 1 dès qu'il a chargé le segment dans DS/SS. Les entrées user (`fa`, `f2`) ne sont pas utilisées, donc elles gardent leur valeur exacte.
- `xp /7xg 0x800` lit 7 mots de 8 octets à l'adresse physique 0x800 : on retrouve nos descripteurs (entrée 0 = 0, puis `00cf9a000000ffff` pour Kernel Code, etc. QEMU affiche chaque mot de 8 octets avec l'octet de poids fort à gauche).

Astuce soutenance
Lance ces deux commandes devant l'évaluateur : l'écran du kernel pour la pile, et `info registers` pour la GDT. Ça prouve les deux points du sujet sans discussion.

12. Questions probables en soutenance

C'est quoi une GDT ?

Un tableau en mémoire où chaque entrée de 8 octets décrit un segment : base, limite, droits (code/data, ring). Le CPU en a besoin en mode protégé pour savoir ce que signifient CS, DS, SS...

Pourquoi la première entrée est vide ?

Intel l'impose. Un sélecteur à 0 provoque une faute si on l'utilise : ça attrape les registres non initialisés.

Pourquoi base 0 et limite 4 Go partout ?

Modèle flat : on neutralise la segmentation (adresse linéaire = offset) parce que la vraie protection mémoire se fera avec la pagination. Linux fait pareil.

Quelle différence entre Kernel Data et Kernel Stack ?

Aucune dans les bits : même base, même limite, mêmes droits. La différence est l'usage : SS pointe sur l'entrée stack, DS sur l'entrée data. Le sujet demande les six entrées, on les a.

Comment tu déclares la GDT ?

Avec `lgdt`, qui charge dans le registre GDTR une structure {limite sur 16 bits, base sur 32 bits}. Ensuite je recharge DS/ES/FS/GS/SS par `mov` et CS par un far jump, sinon le CPU garde les anciens segments de GRUB.

Pourquoi un far jump pour CS ?

CS ne peut pas être écrit avec `mov` : changer CS change le flux d'exécution, donc l'instruction doit aussi dire où continuer. `jmp 0x08:label` fait les deux.

Pourquoi limit = 7*8 - 1 ?

Convention Intel : la limite est l'offset du dernier octet valide, pas la taille.

Pourquoi l'adresse 0x800 ?

Zone de mémoire basse libre (entre la BDA du BIOS à 0x400-0x4FF et le secteur de boot à 0x7C00). Le sujet l'impose ; techniquement n'importe quelle adresse libre et alignée marcherait.

Comment marche ta pile ?

16 Ko réservés dans .bss entre `stack_bottom` et `stack_top`. `esp` démarre à `stack_top` et descend : push = esp - 4 puis écriture, pop = lecture puis esp + 4. `call` pousse l'adresse de retour, `ret` la dépile.

Que fait print_kernel_stack ?

Elle lit esp et ebp avec de l'asm inline, puis affiche la mémoire de esp jusqu'à stack_top, 16 octets par ligne en hexa et en ASCII, avec l'adresse de chaque ligne.

Pourquoi 0xDEADBEEF apparaît comme ef be ad de ?

x86 est little-endian : octet de poids faible en premier.

stdarg.h, c'est pas de la libc ?

Non : c'est un header du compilateur (comme stddef.h, stdint.h), disponible en freestanding. Il n'y a aucune fonction liée derrière, que des builtins gcc.

À quoi sert __attribute__((packed)) ?

À interdire à gcc d'insérer du padding entre les champs : le CPU attend exactement 8 octets dans un ordre précis.

13. Checklist finale

- [ ] GDT de 7 entrées (null + kernel code/data/stack + user code/data/stack) écrite à 0x800
- [ ] `lgdt` exécuté avec limit = 0x37, base = 0x800
- [ ] CS = 0x08, DS/ES/FS/GS = 0x10, SS = 0x18 après `gdt_flush`
- [ ] `info registers` dans QEMU montre `GDT= 00000800 00000037`
- [ ] `printk` fonctionne (%d %u %x %p %s %c)
- [ ] `print_kernel_stack()` affiche esp, ebp, et le dump hexa + ASCII
- [ ] Makefile, linker.ld perso, flags -fno-builtin -fno-stack-protector -nostdlib -nodefaultlibs
- [ ] i386 (-m32, elf_i386)
- [ ] kfs.iso < 10 Mo, présent dans le repo avec le code et le Makefile
- [ ] Tu sais expliquer chaque bit de 0x9A / 0x92 / 0xFA / 0xF2 et le flag 0xC

Sources utiles : OSDev wiki (pages « GDT », « GDT Tutorial », « Segmentation », « Stack »), Intel SDM vol. 3 chapitre 3 (Protected-Mode Memory Management).